
Alcance de la generación respecto de la lógica de negocio

Registro de decisión de arquitectura

Documento **SLCP-ADR-0001** – Versión **1.0**

Estado: **PROPOSED** – pendiente de aprobación humana (TRC-22)

6 de agosto de 2026

Este registro responde a la objeción planteada contra la recomendación D-02 de SLCP-DOC-002, que excluía la generación de lógica de negocio. El análisis concluye que la objeción es parcialmente correcta y que la recomendación original estaba mal formulada: la frontera relevante no separa la lógica de negocio del resto del sistema, sino la lógica especificable de forma declarativa de la lógica algorítmica arbitraria. Se propone en consecuencia una opción nueva, D-02(e), que amplía el alcance de la generación respecto de la propuesta inicial.

Índice

1 Contexto	2
2 Reformulación del problema	2
3 Análisis	2
3.1 Por qué L3 sí debe generarse	3
3.2 Por qué L4 no debe generarse	3
3.3 Límite de la propuesta	4
4 Decisión propuesta	4
5 Consecuencias	4
5.1 Favorables	4
5.2 Desfavorables	4
5.3 Cambios requeridos en SLCP-DOC-001	4
6 Estado y aprobación	5

1 Contexto

SLCP-DOC-002 recomendó para D-02 la opción (c): generar andamiaje, capa de datos, contratos de servicio y CRUD con validaciones, excluyendo la lógica de negocio. La justificación ofrecida entonces fue que derivar lógica de negocio desde lenguaje natural no es verificable. Esa justificación es correcta en su núcleo pero incorrecta en su alcance, porque trata como una sola categoría lo que en realidad son cuatro clases de lógica con propiedades muy distintas frente a la generación automática.

2 Reformulación del problema

La pregunta *por qué no lógica de negocio* carece de respuesta única porque el término agrupa cosas heterogéneas. La Tabla 1 propone una taxonomía operativa basada en un criterio único y verificable: si la lógica admite una especificación declarativa completa, entonces admite generación determinista y trazable; si no la admite, la generación solo puede ser interpretativa.

Tabla 1: Niveles de lógica según su especificabilidad

Nivel	Contenido	Especificación de origen	Generación
L0	Estructura de proyecto, configuración, empaquetado	Descriptor de destino	Determinista
L1	Esquema, migraciones, entidades, repositorios	Modelo de datos	Determinista
L2	Contratos de servicio, CRUD, validaciones sintácticas	Modelo de datos y contrato	Determinista
L3	Invariantes, reglas de decisión, máquinas de estado, cálculos derivados, reglas de autorización, orquestación de flujo	Artefactos declarativos: tablas de decisión, máquinas de estado, expresiones de restricción	Determinista, si y solo si la especificación declarativa existe
L4	Algoritmos propios, optimización, heurísticas, integraciones a medida, lógica dependiente de contexto no formalizado	No existe especificación declarativa completa	Solo interpretativa

Buena parte de lo que en un sistema de gestión institucional se denomina lógica de negocio pertenece al nivel L3 y no al L4. Reglas de matrícula, condiciones de aprobación, cálculos de promedio ponderado, transiciones de estado de un trámite, permisos por rol y validaciones cruzadas son estructuras regulares que admiten especificación declarativa. Excluir las de la generación, como hacía la recomendación original, deja fuera precisamente la porción de valor que justificaría el proyecto.

3 Análisis

3.1 Por qué L3 sí debe generarse

A-01. Es especificable sin ambigüedad. Una tabla de decisión conforme a Obj [1] tiene semántica definida y ejecutable; una máquina de estados tiene transiciones enumerables; una invariante es una expresión booleana sobre el modelo. En los tres casos la traducción a código es una función, no una interpretación, y por tanto es reproducible en el sentido exigido por TRC-23.

A-02. Es trazable de forma natural. Cada regla declarativa es una unidad discreta que enlaza con un requisito individual mediante `implements`, sin las ambigüedades que aparecen cuando un bloque de código imperativo realiza fragmentos de varios requisitos a la vez.

A-03. Permite generar también las pruebas. De una tabla de decisión se derivan automáticamente los casos que cubren todas sus reglas, y de una máquina de estados los que cubren todas sus transiciones. Esto cierra TRC-12 de forma automática en lugar de dejarlo como obligación manual, que es una ventaja considerable y que la recomendación original desaprovechaba.

A-04. Es donde se concentra el cambio. Las reglas de negocio cambian con mucha más frecuencia que la estructura de datos. Mantenerlas como artefacto declarativo versionado, en lugar de disueltas en código imperativo, es beneficioso con independencia de la generación.

3.2 Por qué L4 no debe generarse

B-01. Especificar por completo equivale a programar. Una especificación de lógica arbitraria lo bastante precisa y detallada como para derivar el programa *es* el programa, expresado en otra notación. Brooks [2] distingue entre la complejidad esencial, inherente a la construcción conceptual del software, y la accidental, propia de su representación; la generación de código ataca la accidental y no puede eliminar la esencial. Los intentos históricos de generar comportamiento arbitrario desde modelos terminaron incorporando lenguajes de acción que son, a todos los efectos, lenguajes de programación: el trabajo no desaparece, cambia de sitio.

B-02. El coste de revisión se aproxima al de autoría. La aprobación humana exigida por TRC-22 solo es significativa si quien aprueba comprende lo aprobado. Revisar un repositorio generado es barato porque su forma es predecible y su corrección es estructural. Revisar lógica de negocio generada exige reconstruir mentalmente el razonamiento completo, lo que cuesta aproximadamente lo mismo que escribirla. La ganancia neta tiende a cero, mientras que la apariencia de ganancia incentiva la aprobación sin lectura, que es el peor resultado posible para un sistema cuya propuesta de valor es la trazabilidad verificable.

B-03. La evidencia empírica no respalda la confianza. Las mediciones recientes sobre código generado por modelos de lenguaje muestran una brecha persistente entre corrección aparente y corrección sustantiva: la corrección sintáctica supera el 95%, mientras que la tasa de aprobación en seguridad se mantiene en torno al 55% cuando no se aporta guía explícita [3]. Un artefacto que compila y parece correcto pero es semánticamente erróneo es más peligroso que uno que falla de forma visible, porque atraviesa la revisión superficial.

B-04. Rompe la estrategia de reconciliación. La lógica L4 es exactamente donde las personas necesitan intervenir. Si se genera, debe poder editarse, y entonces la capa separada de D-03 se derrumba y hay que recurrir a regiones protegidas, que se corrompen y convierten el análisis de huérfanos de TRC-11 en ruido inutilizable.

3.3 Límite de la propuesta

La frontera entre L3 y L4 no siempre es nítida y su ubicación depende de la expresividad del lenguaje declarativo adoptado. Un lenguaje de expresiones muy rico desplaza la frontera hacia arriba y, llevado al extremo, se convierte él mismo en un lenguaje de programación, reproduciendo el problema de B-01. La decisión de hasta dónde extender esa expresividad debe tomarse de forma explícita y revisarse en cada iteración, no dejarse crecer por acumulación de casos particulares.

4 Decisión propuesta

Se propone sustituir la recomendación de D-02 por una opción nueva.

D-02(e). La plataforma genera de forma determinista los niveles L0 a L3, incluida la lógica de negocio que admita especificación declarativa completa. La lógica de nivel L4 no se genera: para ella la plataforma produce únicamente la interfaz, el esqueleto de firma y los casos de prueba derivados de los criterios de aceptación del requisito, dejando la implementación al desarrollo humano en la capa manual de D-03.

La diferencia respecto de la recomendación original no es menor: D-02(e) amplía el alcance, no lo restringe. Lo que se conserva de la posición inicial es únicamente el rechazo a derivar comportamiento arbitrario desde texto libre, no el rechazo a generar reglas de negocio.

5 Consecuencias

5.1 Favorables

La cobertura automática de TRC-12 mejora de forma sustantiva, porque las pruebas de las reglas L3 se derivan de la propia especificación declarativa. Las reglas quedan versionadas como artefactos de primer nivel, lo que hace su cambio trazable y su impacto calculable mediante TRC-17. El código generado para L4 incluye pruebas antes que implementación, lo que orienta el trabajo humano hacia un contrato ya definido.

5.2 Desfavorables

El esfuerzo de construcción aumenta de forma apreciable: hay que incorporar un lenguaje o motor de reglas, sus artefactos de intercambio y sus generadores de prueba. La ERS de entrada deja de bastar por sí sola, porque las reglas L3 requieren una especificación estructurada que rara vez está presente en un documento de requisitos convencional; la plataforma deberá asistir su elaboración, lo que introduce un paso intermedio que no existía en el planteamiento original. Además, la frontera L3-L4 exigirá arbitraje humano recurrente.

5.3 Cambios requeridos en SLCP-DOC-001

1. Añadir al vocabulario canónico la entidad `BusinessRule`, con prefijo `RUL`, y sus especializaciones para regla de decisión, invariante y máquina de estados.

2. Añadir un requisito TGT-08 que obligue a generar, junto a cada regla L3, los casos de prueba que cubren la totalidad de sus reglas o transiciones.
3. Añadir un requisito TGT-09 que obligue a generar, para cada elemento L4 identificado, la interfaz y los casos de prueba derivados de los criterios de aceptación, y a marcar el artefacto como de implementación humana pendiente.
4. Incorporar DMN [1] a la tabla de formatos de intercambio de la Sección 8, en dirección de entrada y salida, junto a BPMN [4].
5. Añadir a los criterios de piso la condición de que ninguna regla L3 pueda alcanzar línea base sin su conjunto de pruebas generado y ejecutado.

6 Estado y aprobación

Este registro permanece en estado `PROPOSED`. Conforme a TRC-22, no se incorporará a la línea base ni se aplicarán los cambios enumerados en la Sección 5.3 hasta contar con aprobación humana explícita. Las opciones de respuesta son: aprobar D-02(e) tal como se formula; aprobarla con una frontera L3–L4 distinta de la propuesta; o mantener la recomendación original (c) de SLCP-DOC-002.

Referencias

- [1] *Decision Model and Notation (DMN)*. Object Management Group, Needham, MA, USA, 2026. La versión formal vigente debe verificarse en la página de la especificación, dado que circulan borradores 1.6 y 1.7. <https://www.omg.org/spec/DMN>.
- [2] Frederick P. Brooks. No silver bullet: Essence and accidents of software engineering. *Computer*, 20(4):10–19, 1987. ISSN 0018-9162. doi: 10.1109/MC.1987.1663532.
- [3] Veracode. Spring 2026 GenAI code security update: Despite claims, AI models are still failing security, 2026. Informe de la industria; tasa de corrección sintáctica superior al 95% frente a tasa de aprobación de seguridad cercana al 55%. <https://www.veracode.com/blog/spring-2026-genai-code-security/>.
- [4] *ISO/IEC 19510:2013 Information technology — Object Management Group Business Process Model and Notation*. International Organization for Standardization, Geneva, Switzerland, 2013. <https://www.iso.org/standard/62652.html>.